Communication Theory **2026**

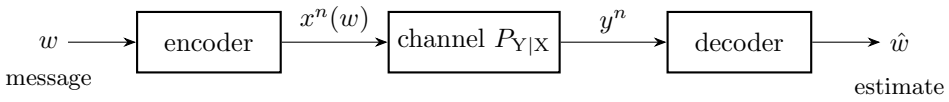
Lecture 1: Channels and maximum-likelihood decoding

Lecturer: S. Yang

Disclaimer: *These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.*

In the demo, a phone recovered a message from a noisy recording. The microphone does not hear bits, it hears numbers. What should the receiver do with them? This lecture answers the question in three steps: what a channel is, what the receiver must keep from what it observed, and what the best possible decoder does with it. A channel code is designed for a channel, and its guarantee only makes sense for that channel, so the channel comes first. Codes come next lecture.

1.1 The setting: channel, code, decoder



Notation. A sequence of length n is written $x^n = (x_1, \ldots, x_n)$. Random variables are upright capitals (X, Y, W) and their values are lowercase (x, y, w). A lowercase p is a probability mass function or a density, and a capital $P(\cdot)$ is the probability of an event. These are the notations of the Information theory course.

A **channel** $P_{Y|X}$ has an input alphabet $\mathcal{X}$, an output alphabet $\mathcal{Y}$ and a transition law $p(y \mid x)$: the probability of observing y when x is sent. It is a probability mass function for discrete y and a density for continuous y, and we shall not need more than these two cases.[1] Over n uses, the channel is **memoryless** if

$$p(y^n \mid x^n) = \prod_{i=1}^{n} p(y_i \mid x_i). \tag{1.1}$$

The transmitter wants to send a **message** $w \in \{1, \ldots, M\}$. An **encoder** maps each message w to a **codeword** $x^n(w) \in \mathcal{X}^n$, and different messages have different codewords. The **code** of length n is the set of codewords, $\mathcal{C} = \{x^n(1), \ldots, x^n(M)\} \subseteq \mathcal{X}^n$. Its **rate** is

$$R := \frac{\log_2 M}{n} \quad \text{bits per channel use.}$$

© 2026 Sheng Yang. All rights reserved.

[1] The general setting, with measures and kernels, is in the supplement "Probability measures, kernels and laws" of the Information theory course.

For a binary code that carries k information bits, the message is the bit sequence u^k, so $M = 2^k$ and $R = k/n$.

A **decoder** is a map $y^n \mapsto \hat{w}(y^n) \in \{1, \dots, M\}$. Throughout the course the message W is uniform on $\{1, \dots, M\}$. The decoder makes an error when its estimate is not the message that was sent, and its error probability is

$$P_e := P\big(\hat{\mathrm{W}} \neq \mathrm{W}\big), \qquad \hat{\mathrm{W}} = \hat{w}(\mathrm{Y}^n).$$

This is the probability of a wrong *message*. Since the encoder is one-to-one, choosing a message is choosing a codeword, and we also write the decision as $\hat{x}^n = x^n(\hat{w})$. So P_e depends on the set $\mathcal{C}$ and on the decoder, not on which message is given which codeword. The fraction of wrong information bits is another quantity, and Lecture 2 compares the two.

Proposition (the best decoder). When the message is uniform, the error probability is minimised by the **maximum-likelihood** (ML) decoder

$$\hat{w}(y^n) = \arg \max_{w \in \{1,\dots,M\}} p\big(y^n \mid x^n(w)\big), \qquad \text{that is,} \qquad \hat{x}^n = \arg \max_{x^n \in \mathcal{C}} p(y^n \mid x^n). \tag{1.2}$$

Proof. For any decoder, sort the outputs y^n by the decision they lead to. The probability of a correct decision is

$$\begin{aligned} 1 - P_e &= \sum_{w=1}^{M} \frac{1}{M} \sum_{y^n \colon \hat{w}(y^n) = w} p\big(y^n \mid x^n(w)\big) = \frac{1}{M} \sum_{y^n} p\big(y^n \mid x^n(\hat{w}(y^n))\big) \\ &\leq \frac{1}{M} \sum_{y^n} \max_{x^n \in \mathcal{C}} p(y^n \mid x^n), \end{aligned}$$

with equality for the ML decoder (the sums over y^n are integrals for continuous outputs). □

If several codewords reach the maximum, any of them can be chosen. If the message were not uniform, the best decoder would maximise $P(\mathrm{W} = w)\, p(y^n \mid x^n(w))$ instead; with a uniform message the first factor is the same for all w. For a memoryless channel, taking the logarithm of (1.1) turns the product into a sum:

$$\hat{x}^n = \arg \max_{x^n \in \mathcal{C}} \sum_{i=1}^{n} \ln p(y_i \mid x_i). \tag{1.3}$$

What the receiver should keep. In (1.3), the observation y_i enters only through the **likelihood function** $a \mapsto p(y_i \mid a)$, $a \in \mathcal{X}$. Multiplying this function by a positive factor that depends only on y_i shifts the scores of all codewords by the same amount, so the decoder needs it only up to such a factor.

So a good receiver hands the decoder, for each channel use, $|\mathcal{X}|$ numbers defined up to a common factor. From now until Lecture 3 the input is binary. Larger alphabets come back with Reed–Solomon codes in Lecture 4.

1.2 Binary input: one number per bit

Let the input be a bit $x \in \{0,1\}$. The likelihood function has two values, $p(y \mid 0)$ and $p(y \mid 1)$. Up to a factor, it is one number, their ratio.

Definition. The **log-likelihood ratio** (LLR) of an observation y is

$$L(y) := \ln \frac{p(y \mid 0)}{p(y \mid 1)} \in [-\infty, +\infty]. \tag{1.4}$$

One of the two likelihoods can be zero. Then $L = +\infty$ or $L = -\infty$, with the conventions $\ln(a/0) = +\infty$ and $\ln(0/a) = -\infty$ for $a > 0$. They cannot both be zero, because such a y is never observed. With an infinite LLR we use only $e^{-\infty} = 0$, $e^{+\infty} = +\infty$, $1/(+\infty) = 0$ and sums of non-negative terms, and we never subtract two infinite numbers.

The sign of L says which bit is more likely, $L > 0$ for 0 and $L < 0$ for 1, and $|L|$ says how sure we are. If the bit X is uniform, Bayes' rule gives

$$P(\mathrm{X} = 0 \mid \mathrm{Y} = y) = \frac{p(y \mid 0)}{p(y \mid 0) + p(y \mid 1)} = \frac{1}{1 + e^{-L(y)}}.$$

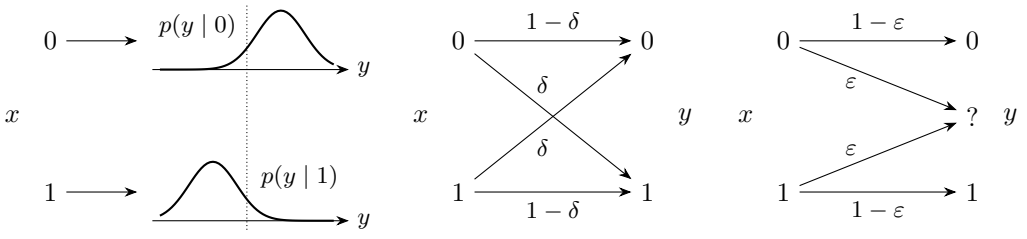
For instance, $L = 0$ means "no idea" $(1/2)$, $L = 1$ gives 0.73, $L = 3$ gives 0.95 and $L = +\infty$ means "0 for sure".

Let $\hat{y}$ be the more likely bit, called the **hard decision**: $\hat{y} = 0$ if $L > 0$ and $\hat{y} = 1$ if $L < 0$ (if $L = 0$, either value will do). Its likelihood $p(y \mid \hat{y})$ is positive, and the likelihood of the other bit is $e^{-|L|}$ times as large. In one formula,

$$p(y \mid x) = \underbrace{p(y \mid \hat{y})}_{\text{does not depend on } x} \times \begin{cases} 1 & \text{if } x = \hat{y}, \\ e^{-|L(y)|} & \text{if } x \neq \hat{y}, \end{cases} \qquad x \in \{0,1\}. \tag{1.5}$$

The first factor is the same for both values of the bit. Everything that distinguishes 0 from 1 is in the LLR: its sign names the more likely bit, and $|L|$ is the price of contradicting it. The formula also holds when $|L| = \infty$: the other bit is then impossible.

1.3 Three binary-input channels

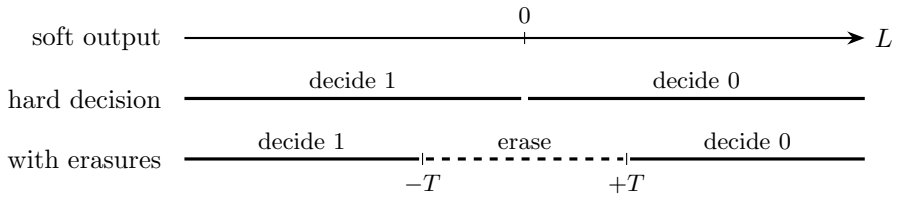


Three channels, three kinds of LLR.

- **Continuous output**: y is a real number, with a density $p(y \mid 0)$ when 0 is sent and a density $p(y \mid 1)$ when 1 is sent. Then $L(y)$ can be any real number: each bit comes with its own confidence. This is the channel as the receiver first sees it, and we call it a **soft-output channel**.
- **Binary symmetric channel** BSC(δ): $y \in \{0, 1\}$, and each bit is flipped with probability δ. Then $L(0) = +\ell_\delta$ and $L(1) = -\ell_\delta$ with $\ell_\delta := \ln \frac{1-\delta}{\delta}$. Every bit comes with the same confidence.
- **Binary erasure channel** BEC(ε): $y \in \{0, 1, ?\}$, and each bit is erased with probability ε, otherwise it is received correctly. Then $L(0) = +\infty$, $L(1) = -\infty$ and $L(?) = \ln \frac{\varepsilon}{\varepsilon} = 0$. A bit is either known or completely unknown.

For example, $\delta = 0.1$ gives $\ell_\delta = \ln 9 \approx 2.2$.

The three channels can describe the same physical link, seen through three receivers. The first one is the original, and the other two are what remains of it when the receiver keeps less. Suppose the receiver has computed the LLR $L(y)$ of each bit.



A receiver that keeps only the sign of L passes on the hard decision $\hat{y}$ of each bit. Some decisions are wrong, and the decoder sees flipped bits: this is the BSC. A more careful receiver refuses to decide when $|L|$ is below a threshold T and declares an erasure. The BEC is the idealised version of this receiver, in which the bits that are kept are all correct. A receiver that passes L on keeps everything.

The decoder needs only $L(y)$, so we may as well say that a soft-output channel delivers the LLR itself, $y_i = L_i$. This description needs no assumption on the noise. In Lectures 7 and 8 we shall see how a real receiver computes them. For example, if the bit is sent as the level $\pm\sqrt{E_s}$, that is, $y = (-1)^x\sqrt{E_s} + z$ with Gaussian noise $z \sim \mathcal{N}(0, N_0/2)$, then $L(y) = 4\sqrt{E_s}\, y/N_0$.

Example 1.1 (what we observe is never binary)**. Flash memory.** A cell stores a bit as an electric charge, and the charge sets the voltage at which the cell starts to conduct. This voltage is a real number, and it moves with time and wear. A read compares it with one reference voltage and returns a bit: a hard decision, fast, and good enough while the memory is new. When the decoder fails, the controller reads the cell again with other reference voltages. It then knows in which interval the voltage lies, and this gives an LLR with several levels. A hard disk is similar: the read head delivers a voltage, and the bits come after a decision.

QR code. The camera gives a grey level for each small square. The reader compares it with a threshold and keeps "black" or "white": again a hard decision. A square under a stain or a logo is unreliable, and a reader that knows where it is can declare an erasure (TP2).

In both cases the physical observation is continuous. The binary output is a choice of the receiver, made for convenience: simpler circuits, less to store, a simpler decoder. Example 1.4 shows what this choice can cost.

Remark 1.2 (every channel is a soft-output channel)**.** Take any binary-input channel and let the receiver output $\ell = L(y)$ instead of y. By (1.5), the decoder loses nothing. And the new output $\mathrm{L} = L(\mathrm{Y})$ is its own LLR: for a discrete output and a finite ℓ,

$$P(\mathrm{L} = \ell \mid \mathrm{X} = 0) = \sum_{y:\, L(y)=\ell} p(y \mid 0) = \sum_{y:\, L(y)=\ell} e^{\ell}\, p(y \mid 1) = e^{\ell}\, P(\mathrm{L} = \ell \mid \mathrm{X} = 1),$$

and $\mathrm{L} = +\infty$ can occur only when 0 is sent, $\mathrm{L} = -\infty$ only when 1 is sent. So "the channel outputs LLRs" is a consistent definition, and it covers every binary-input memoryless channel. The BSC and the BEC are the special cases in which L takes two and three values.

1.4 Maximum-likelihood decoding with LLRs

A binary code is a set $\mathcal{C} \subseteq \{0,1\}^n$. A codeword is $x^n = (x_1, \ldots, x_n)$, and $L_i = L(y_i)$ is the LLR of the i-th observation. Let $\hat{y}_i$ be the hard decision on bit i, as in Section 1.2.

Proposition (ML with LLRs). For a binary code on a memoryless channel,

$$\hat{x}^n = \arg\min_{x^n \in \mathcal{C}} \sum_{i:\, x_i \neq \hat{y}_i} |L_i|. \tag{1.6}$$

If all the LLRs are finite, this is also

$$\hat{x}^n = \arg\max_{x^n \in \mathcal{C}} \sum_{i=1}^{n} (-1)^{x_i} L_i. \tag{1.7}$$

Proof. Put (1.5) in (1.1):

$$p(y^n \mid x^n) = \prod_{i=1}^{n} p(y_i \mid \hat{y}_i) \;\times\; \exp\Bigl(-\sum_{i:\, x_i \neq \hat{y}_i} |L_i|\Bigr).$$

The product is positive and does not depend on x^n, so the likelihood is largest where the sum is smallest. This sum has non-negative terms, so it is well defined even with infinite LLRs: it is $+\infty$ exactly when the codeword is impossible. If all the LLRs are finite, $(-1)^{x_i} L_i$ is $+|L_i|$ if $x_i = \hat{y}_i$ and $-|L_i|$ if $x_i \neq \hat{y}_i$, so

$$\sum_{i=1}^{n} (-1)^{x_i} L_i = \sum_{i=1}^{n} |L_i| \;-\; 2 \sum_{i:\, x_i \neq \hat{y}_i} |L_i|,$$

and the first sum does not depend on x^n. □

The cost $\sum_{i:\,x_i \neq \hat{y}_i} |L_i|$ counts the disagreements between the codeword and the hard decisions, and each disagreement is weighted by its reliability: contradicting a confident bit is expensive, contradicting a doubtful bit is cheap. It is a **weighted Hamming distance**. Let us read (1.6) on the three channels.

BSC. Let $d_H(y^n, x^n)$ be the **Hamming distance** between y^n and x^n, that is, the number of positions where they differ. If $\delta < \frac{1}{2}$, the hard decisions are the received bits, $\hat{y}_i = y_i$, and all weights are equal, $|L_i| = \ell_\delta$. The cost is $\ell_\delta \, d_H(y^n, x^n)$, and ML is **minimum Hamming distance decoding**: pick the codeword nearest to y^n. If $\delta > \frac{1}{2}$, then $\ell_\delta < 0$, the hard decisions are the flipped bits, and ML picks the codeword *farthest* from y^n. This is not a paradox: flip all received bits and the channel is a BSC$(1-\delta)$.

BEC. An unerased bit has an infinite weight and an erased bit has weight 0. A codeword that disagrees with an unerased bit has cost $+\infty$: it is impossible. All codewords that agree with every unerased bit have the same cost, 0. So ML is: *keep the codewords that are consistent with what was received.* If exactly one remains, it is the right one. If several remain, the decoder can only guess among them.

Soft output. Every bit has its own weight, and nothing simplifies: ML is **minimum weighted Hamming distance decoding**. Two errors on doubtful bits can cost less than one error on a confident bit, which a hard-decision decoder cannot see.

Remark 1.3 (the picture with ± 1)**.** Suppose that all the LLRs are finite, and send each codeword to the point $s^n \in \{+1, -1\}^n$ with $s_i = (-1)^{x_i}$. The sum in (1.7) is the correlation $\sum_i s_i L_i$ between this point and the LLRs. Since $\|s^n\|^2 = n$ for every codeword, for any $\alpha > 0$

$$\|L^n - \alpha s^n\|^2 = \|L^n\|^2 - 2\alpha \sum_{i=1}^{n} s_i L_i + \alpha^2 n,$$

so ML also picks the point nearest to the sequence of LLRs $L^n = (L_1, \ldots, L_n)$, up to scale: **minimum Euclidean distance decoding**. This picture comes back in Lectures 7 and 8, where ± 1 are the transmitted levels.

Example 1.4 (one reception, three receivers)**.** Consider the code

$$\mathcal{C} = \{00000,\ 01011,\ 10101,\ 11110\},$$

with $n = 5$ and $M = 4$ (two information bits), so $R = 2/5$. The codeword 00000 is sent and the receiver computes the LLRs

$$L^n = (+4,\ -1,\ +3,\ -1,\ +6).$$

Two LLRs have the wrong sign, but both are small. Three receivers look at them. The first passes L^n on. The second makes hard decisions and passes $\hat{y}^n = 01010$ on. The third erases the bits with $|L_i| \leq 1$ and passes 0 ? 0 ? 0 on.

codeword x^n	soft: $\sum_{i:\,x_i \neq \hat{y}_i} \lvert L_i \rvert$	hard: $d_H(\hat{y}^n, x^n)$	erasures: consistent?
00000	**2**	2	**yes**
01011	6	**1**	no
10101	15	5	no
11110	7	2	no

The soft decoder finds 00000, and so does the decoder with erasures. The hard-decision decoder sees $\hat{y}^n = 01010$, which is nearer to 01011: it picks the wrong codeword. For the

soft decoder, 00000 contradicts two doubtful bits (cost $1+1$), while 01011 contradicts one confident bit (cost 6). Each decoder is the best one for what it was given. What differs is what the receiver kept.

1.5 A first code: repetition

The $(n, 1)$ **repetition code** has two codewords, $00\cdots0$ and $11\cdots1$, and rate $R = 1/n$. In (1.6) the cost of $00\cdots0$ is the sum of the $|L_i|$ over the negative LLRs, and the cost of $11\cdots1$ is the sum over the positive ones. So ML decides 0 if the positive LLRs weigh more than the negative ones, that is, with finite LLRs, if $\sum_i L_i > 0$:

- BSC: majority vote;
- BEC: copy any unerased bit; guess only if all n bits are erased, which has probability ε^n;
- soft output: add the LLRs and look at the sign.

For $n = 3$ on the BSC, the decoder fails when 2 or 3 bits are flipped:

$$P_e = 3\delta^2(1-\delta) + \delta^3 = 3\delta^2 - 2\delta^3.$$

With $\delta = 0.1$, this is 0.028 instead of 0.1, and the price is a rate of $1/3$. Longer repetition codes drive P_e to zero, but the rate $1/n$ goes to zero as well.

Is this trade unavoidable? No, and this is the promise of the introduction. The BSC(0.1) has capacity $C \approx 0.53$ bit per channel use: there are codes of rate $1/2$, much longer and much smarter than repetition, whose error probability is as small as we wish. We take this as a fact until Lecture 11. The next lectures build such codes and their decoders.

1.6 What a code guarantees: the minimum distance

Definition. The **minimum distance** of a code $\mathcal{C}$ is

$$d := \min_{x^n \neq \tilde{x}^n \in \mathcal{C}} d_H(x^n, \tilde{x}^n).$$

The repetition code has $d = n$. The code of Example 1.4 has $d = 3$.

Theorem (guarantees from d).

1. On the BSC, minimum distance decoding corrects every pattern of up to $\lfloor (d-1)/2 \rfloor$ errors.
2. On the BEC, ML decoding corrects every pattern of up to $d-1$ erasures.

Proof. 1. Let x^n be sent and let y^n differ from x^n in $t \le (d-1)/2$ positions. The Hamming distance satisfies the triangle inequality. So for any other codeword $\tilde{x}^n$,

$$d_H(y^n, \tilde{x}^n) \;\ge\; d_H(x^n, \tilde{x}^n) - d_H(x^n, y^n) \;\ge\; d - t \;>\; t = d_H(y^n, x^n),$$

because $2t \le d-1 < d$. The sent codeword is strictly the nearest.

2. Suppose at most $d-1$ bits are erased and two different codewords are consistent with the unerased bits. They can differ only in erased positions, hence in at most $d-1$ positions. This contradicts the definition of d. So exactly one codeword is consistent, the one that was sent. □

Both guarantees are the best possible. Take two codewords $x^n, \tilde{x}^n$ at distance d. If x^n is sent and $\lfloor (d-1)/2 \rfloor + 1$ of the positions where they differ are flipped, y^n is at least as near to $\tilde{x}^n$ as to x^n. If the d positions where they differ are erased, both are consistent.

On the BEC a code corrects about twice as many erasures as it corrects errors on the BSC: knowing *where* the channel hit is worth a lot. For the code of Example 1.4, $d = 3$ guarantees one error or two erasures. In the example two hard decisions were wrong, so the hard decoder had no guarantee, while the two erasures were within reach.

On a soft-output channel there is no such deterministic guarantee, because any number of LLRs can point the wrong way. Lecture 2 gives a probabilistic guarantee with the same d, and asks what distance costs in rate.

To remember.

- A message $w \in \{1, \dots, M\}$ is sent as the codeword $x^n(w)$; $R = (\log_2 M)/n$, and $P_e = P(\hat{\mathrm{W}} \neq \mathrm{W})$ is the probability of a wrong message.
- With a uniform message, the best decoder is ML: $\hat{x}^n = \arg\max_{x^n \in \mathcal{C}} p(y^n \mid x^n)$. On a memoryless channel it needs, for each channel use, the likelihood function $a \mapsto p(y_i \mid a)$ up to a factor.
- Binary input: one number per bit, the LLR $L = \ln \frac{p(y|0)}{p(y|1)}$. BSC: $L = \pm \ln \frac{1-\delta}{\delta}$. BEC: $L \in \{+\infty, 0, -\infty\}$. Soft output: any real L.
- ML $= \arg\min_{x^n \in \mathcal{C}} \sum_{i:\, x_i \neq \hat{y}_i} |L_i|$, where $\hat{y}_i$ is the hard decision: each disagreement costs its reliability. Hamming distance on the BSC, consistency on the BEC, weighted Hamming distance on a soft-output channel (equivalently, the nearest point ± 1 to L^n).
- The minimum distance d guarantees $\lfloor (d-1)/2 \rfloor$ errors on the BSC and $d-1$ erasures on the BEC.

Exercises

Exercise 1.1 (an asymmetric channel)**.** A binary-input channel has output $y \in \{0, 1\}$ with $p(1 \mid 0) = 0$ and $p(0 \mid 1) = \gamma$, where $0 < \gamma < 1$: a 1 is sometimes read as a 0, a 0 is never

read as a 1. Compute $L(0)$ and $L(1)$. Describe the ML decoder of the $(n, 1)$ repetition code on this channel, and compute its error probability.

Exercise 1.2 (repetition, $n = 5$)**.** Compute the error probability of the $(5, 1)$ repetition code with ML decoding on the BSC(0.1). Compare with $n = 3$. What is the probability that the decoder has to guess on the BEC(0.1)?

Exercise 1.3 (hard against soft)**.** The $(3, 1)$ repetition code is used and the receiver computes $L^n = (+3, -1, -1)$. What does the majority vote decide? What does ML decide? Compute $P(\mathrm{X} = 0 \mid \mathrm{Y}^n = y^n)$ for a uniform bit. (Hint: by (1.1), the LLR of the whole observation is $\sum_i L_i$.)

Exercise 1.4 (three receivers again)**.** With the code of Example 1.4, the receiver computes $L^n = (-3,\ +2,\ +1,\ -1,\ -5)$. Decode with the soft decoder, with hard decisions, and with erasures where $|L_i| \le 1$.

Exercise 1.5 (detection)**.** Show that a code of minimum distance d **detects** every pattern of 1 to $d - 1$ errors on the BSC: the received word is not a codeword. Why can a pattern of d errors go unnoticed?